Alen Da Luz 28/04/2026

Scheduling della CPU

1. Mono-Tasking

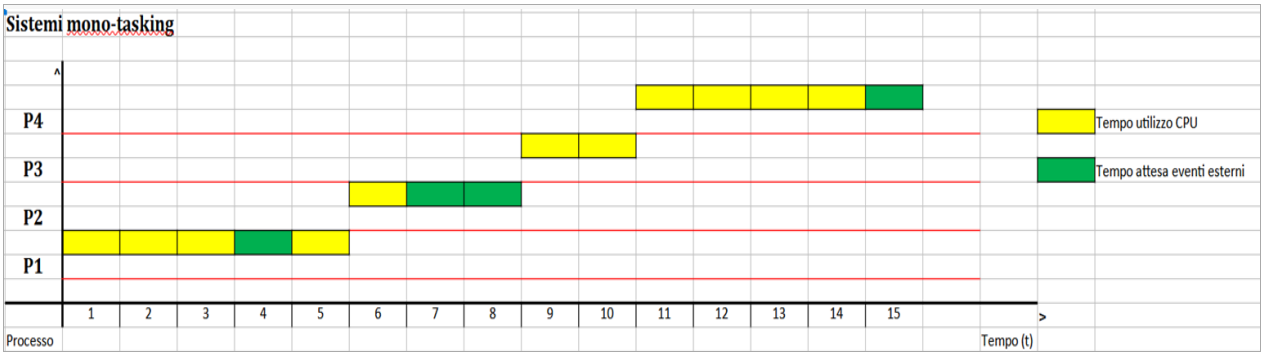


Fig. 1 — Sistemi Mono-tasking

2. Multi-Tasking

Fig. 2 — Sistemi Multi-Tasking

3. Time-Sharing

Fig. 3 — Sistemi Time-Sharing

4. Analisi Comparativa tra Sistemi Multi-tasking e Time-sharing

L'evoluzione delle architetture software ha sancito il definitivo superamento dei sistemi **mono-tasking**, ormai considerati obsoleti in favore di soluzioni significativamente più efficienti. Tra queste, i sistemi **multi-tasking** e **time-sharing** rappresentano i pilastri della gestione moderna dei processi, distinguendosi principalmente per le **modalità di allocazione delle risorse della CPU** e per i relativi tempi di attesa.

5. Differenze Operative e Gestione dei Tempi

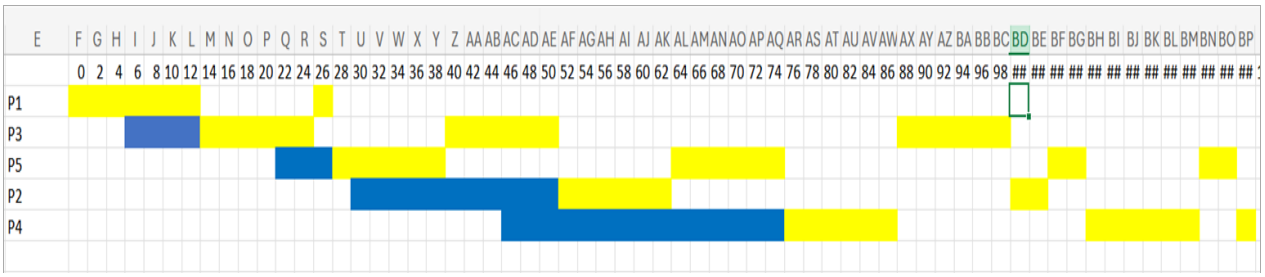
La distinzione fondamentale tra le due metodologie risiede nel **criterio di interruzione dei processi in esecuzione**:

- ***Sistemi Multi-tasking***: In questo scenario, la gestione è orientata al completamento o alla sospensione naturale del compito. Se un processo richiede, come da esempio, tre secondi per l'elaborazione, esso manterrà il controllo della CPU in modo continuativo, a meno che non intervenga un **evento esterno** (come la richiesta di input/output) o la necessità di eseguire un processo con **priorità superiore**. Di conseguenza, eventuali altri processi in coda rimarranno in stato di attesa per l'intera durata dell'esecuzione del primo.
- ***Sistemi Time-sharing***: Questa architettura introduce il concetto di **quantum temporale**, ovvero una **frazione di tempo predefinita e assegnata a priori** a ciascun processo. Riprendendo l'esempio precedente, se il quantum è fissato a un secondo, l'esecuzione del primo processo verrà sospesa allo scadere del tempo stabilito per consentire l'avanzamento di un secondo processo. Questo meccanismo di avvicendamento ciclico garantisce una **distribuzione equa delle risorse** e riduce drasticamente i **tempi di latenza percepiti**.

6. Analisi Tecnica della Sequenza di Scheduling

Processo	Tempo di esecuzione	Tempo di attesa	Tempo di esecuzione dopo attesa
P1	3 secondi	1 secondo	1 secondo
P2	1 secondo	2 secondi	—
P3	2 secondi	—	—
P4	4 secondi	1 secondo	—

Seguendo le impostazioni della tabella andiamo a riprodurre la **sequenza di Scheduling** di 5 processi.



Scheduling Processi - Report Compatto

TS	Inizio	Fine	Processo	Coda a fine TS (←)
1	0	12	P1	P3(6*)
2	12	24	P3	P1(12) P5(22*)
3	24	26	P1 END	P5(22) P3(24)
4	26	38	P5	P3(24) P2(30*)
5	38	50	P3	P2(30) P5(38) P4(46*)
6	50	62	P2	P5(38) P4(46) P3(50)
7	62	74	P5	P4(46) P3(50) P2(62)
8	74	86	P4	P3(50) P2(62) P5(74)
9	86	98	P3	P2(62) P5(74) P4(86)
10	98	102	P2 END	P5(74) P4(86) P3(98)
11	102	106	P5 END	P4(86) P3(98)
12	106	118	P4	P3(98)
13	118	122	P3 END	P4(118)
14	122	124	P4 END	—

(*) Nuovo processo in coda.

L'algoritmo **Round Robin** implementato evidenzia un'**equa distribuzione della CPU**, ma introduce un inevitabile **overhead di latenza** per i processi a bassa priorità o arrivati tardivamente. La natura preemptiva costringe processi brevi (come P1) ad attendere il completamento dei time slice altrui, aumentando il **Wait Time complessivo**. Il tempo di partenza risulta critico: P4 e P5 subiscono un ritardo significativo prima del primo accesso alle risorse, **penalizzando la reattività del sistema**. In conclusione, sebbene il Round Robin **garantisca l'assenza di starvation**, l'efficienza globale è strettamente legata alla dimensione del quantum.